Coding scheme diagrams

Pipeline, codeword construction, and the block-circulant decoder

---

Five figures: the shared experimental pipeline, how each scheme builds its codeword, and the two block-circulant figures that explain why an optimal code can still decode poorly. Everything here is either a pipeline the report has to describe anyway, or the structure behind a measured result. Earlier drafts also carried worked numeric examples and algorithm flowcharts; those were teaching aids rather than results and have been dropped.

Figures are drawn from this project's own code (`1D_RS_Galois.py`, `2D_RS_Galois.py`, `BCcodeEthereum.py`). The BC figures follow the published construction — Sasidharan, Viterbo & Hoang Dau, *"Block Circulant Codes with Application to Decentralized Systems,"* IEEE Trans. Commun. 73(10), 2025 (arXiv:2406.12160) — since that is what the implementation is meant to realise.

Experimental pipeline

Every number in the benchmark report comes from running this loop once per trial. The blob is fixed and the erasure rate is swept; at each rate the codeword is corrupted at randomly chosen positions and handed to the scheme's own decoder.

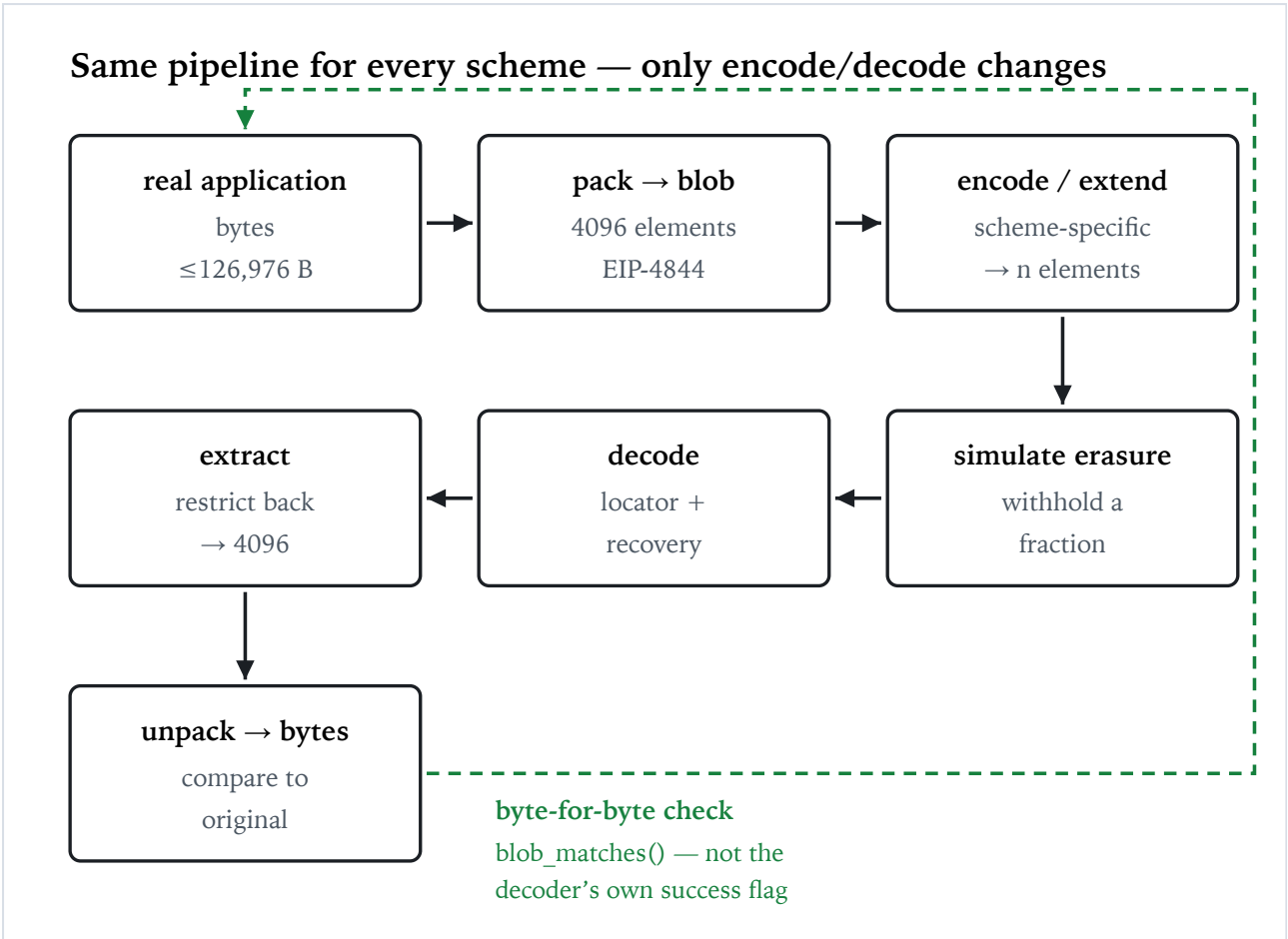


Fig. 1. The measurement pipeline. Real application bytes are packed into one EIP-4844 blob (4096 field elements, 31 usable bytes each), the scheme under test expands that blob into a longer codeword, a fraction of the codeword is withheld to stand in for samples peers failed to serve, the decoder tries to rebuild it, and the result is unpacked back to bytes. Only *encode/extend* and *decode* differ between schemes; the input, the erasure model and the test of success are byte-identical for all of them, which is what makes a difference in the results attributable to the code rather than to the setup. The dashed return path is the success test itself: the recovered bytes must equal the original bytes exactly. It deliberately does not ask the decoder whether it succeeded — one decoder was found reporting success on data it had silently corrupted, and only this comparison caught it.

How each scheme builds its codeword

The two Reed-Solomon variants share one idea applied at different dimensions. Both figures lead with that idea rather than with the transform steps, because the mechanism is easier to follow once the purpose is clear.

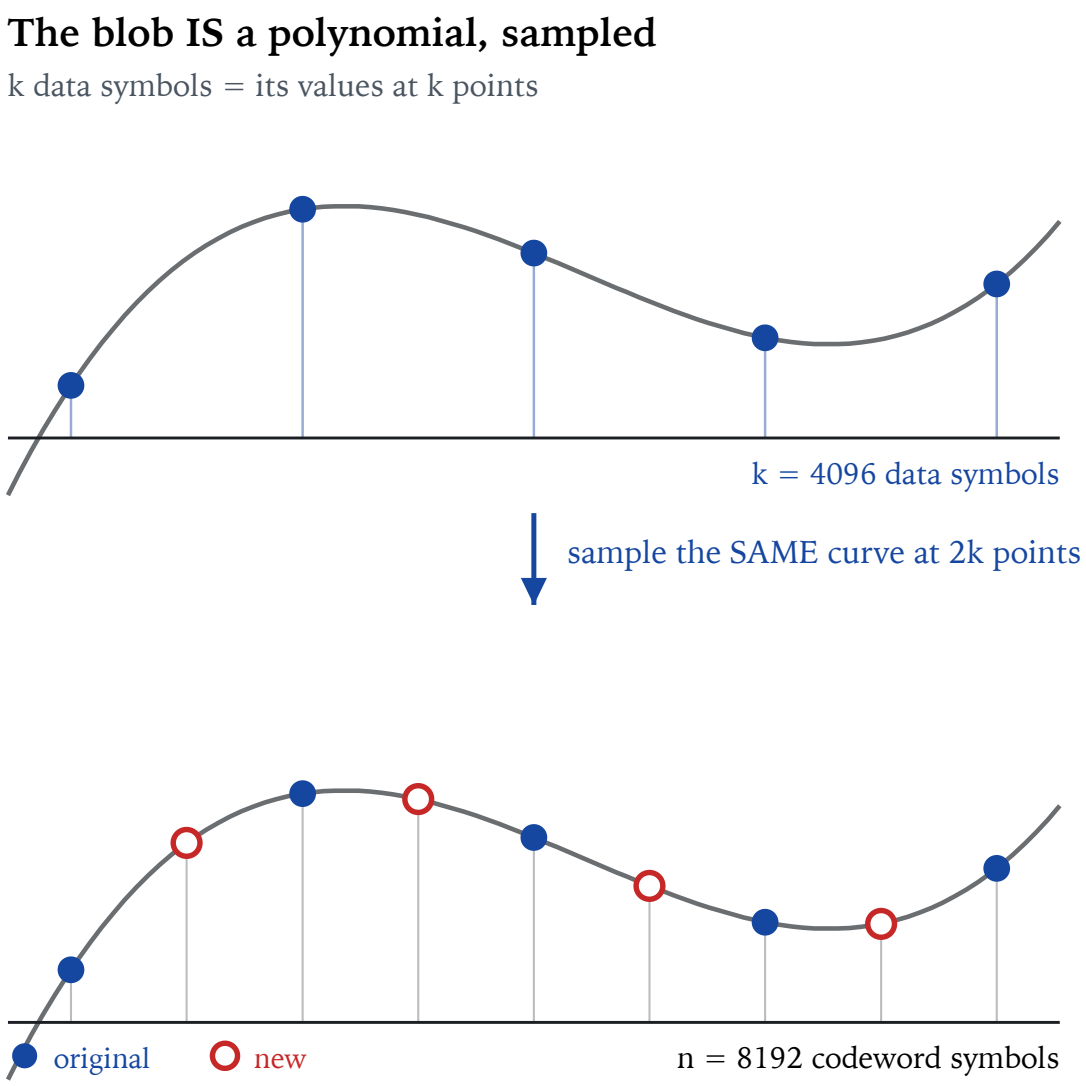


Fig. 2. 1D-RS. The blob's k field elements are not raw bytes to be copied; they are the *values of a single polynomial* sampled at k points. Encoding finds that polynomial and samples it at n = 2k points instead. Because a degree-<k polynomial is pinned down by any k of its values, any k surviving symbols reconstruct the curve and hence every original symbol. 2× overhead, and the reason 1D-RS reaches its theoretical limit exactly.

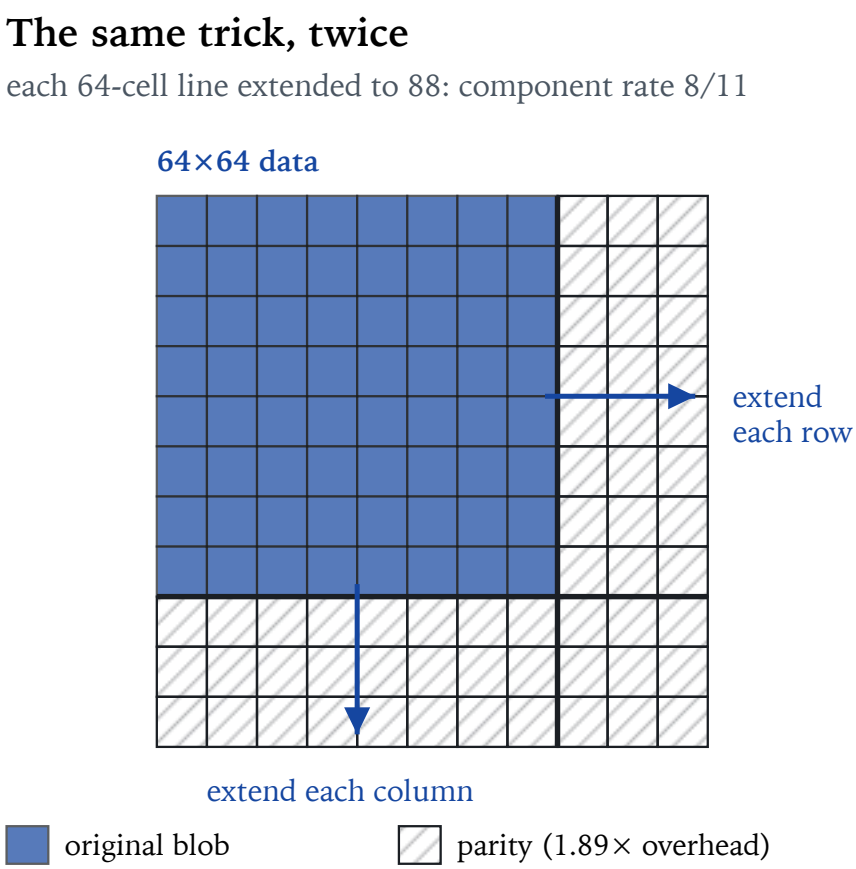


Fig. 3. 2D-RS. The blob is reshaped into a 64×64 grid and the 1D-RS step is applied twice: once to extend every row, then once to extend every column of the result. Every row and every column of the outcome is independently a 1D-RS codeword, so a lost cell can be repaired from whichever of its two lines has enough survivors, which is what lets the iterative decoder cascade. The extension is 64→88 per axis rather than the 64→128 of earlier drafts: a product code's rate is the *product* of its component rates, so doubling both axes costs 4× storage and made the comparison against two 2×-overhead schemes unfair. Component rate 8/11 per axis gives 1.89× overall.

Block-circulant structure

Block-circulant codes replace one global polynomial with many small local ones. How much they overlap is the parameter that decides whether the resulting code is any good.

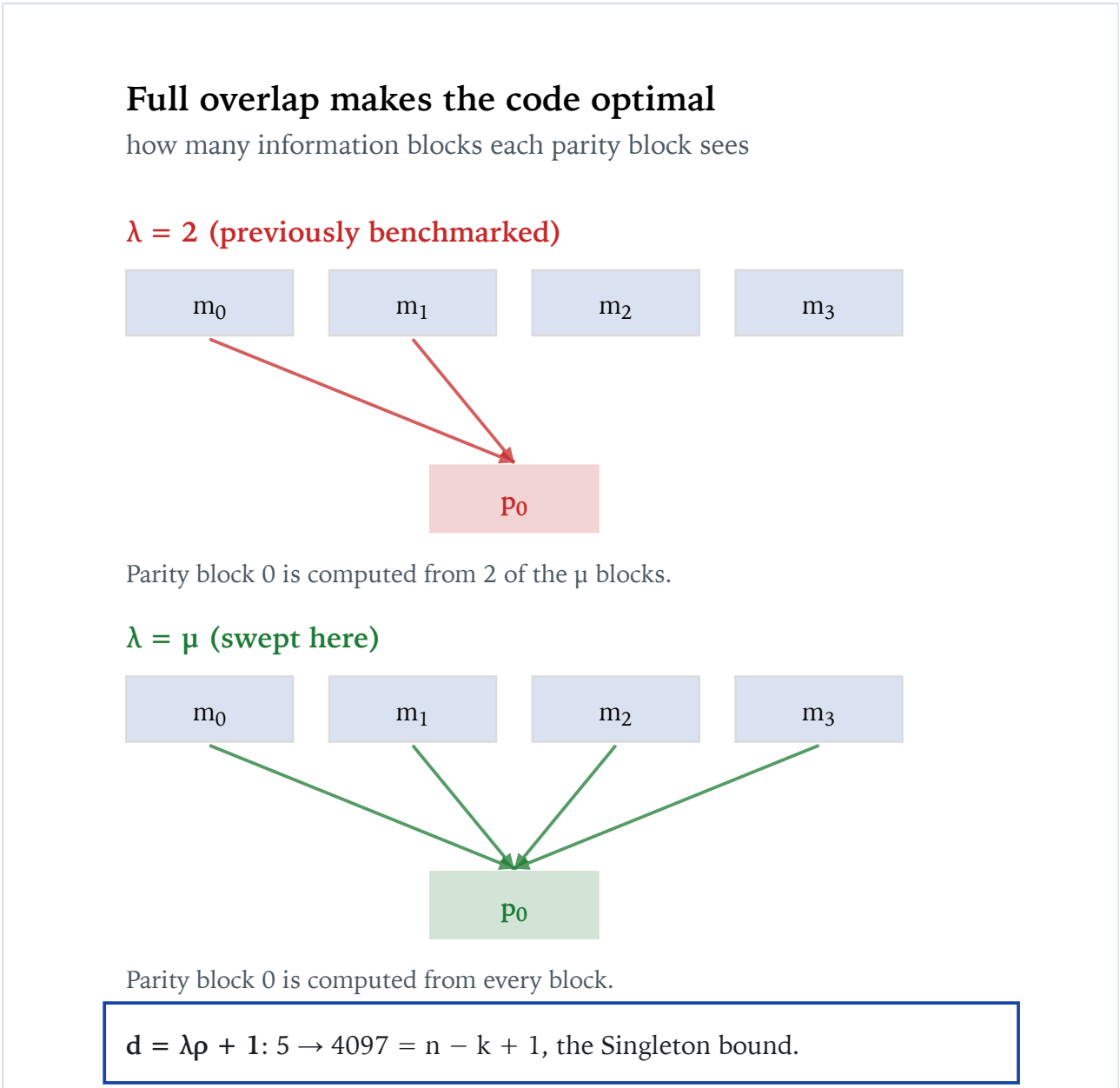


Fig. 4. Overlap decides everything. Minimum distance is $d = \lambda\rho + 1$, so the λ=2 parameterisation benchmarked earlier is *guaranteed* to correct only four erasures in an 8192-symbol codeword; anything beyond that depended on how the erasures happened to fall. Setting $\lambda = \mu$ makes every parity block depend on every information block and lifts d to 4097 $= n - k + 1$, the Singleton bound. As codes, the parameterisations swept in the report are therefore MDS — exactly as good as Reed-Solomon.

Why block-circulant tolerance is limited

The parameterisations swept in the report are MDS codes: any 4096 surviving symbols determine the blob, exactly as for 1D-RS. They nevertheless stop recovering far earlier, and the reason is entirely in the decoder, which never looks beyond a single local block.

The code is optimal; the decoder is local

what one local block can see, and what it can fix

one local block's support (λ = μ)



$n_0 = \rho + \lambda\omega = \rho + 4096$

It solves when its support holds at most ρ erasures.
That support is the whole message, so **one block** finishes the job.

$4096x + \rho x \le \rho$
so the decoder gives out at $x = \rho/n_0$ — not at 0.50.

ω = ρ	n_0	ρ/n_0	solve cost
1024	5120	0.200	1024×1024
512	4608	0.111	512×512
256	4352	0.059	256×256

The gap is the decoder, not the code.
All three are MDS: any 4096 surviving symbols suffice.
Bigger blocks close more of it; the solve stays $O(\rho^2)$.

Fig. 5. Why the measured tolerance is far below 0.50 anyway. `DecCBC[2]` solves one block at a time, and a block succeeds only when its own support holds at most ρ erasures. Because λ = μ, that support is the entire message plus the block's own parity, which cuts both ways: one successful block recovers the whole blob at once, but the erasure budget ρ has to cover erasures spread across all 4096 message symbols as well. Solving $4096x + \rho x \le \rho$ gives $x \le \rho/n_0$, which is what the measurements land on. Larger local blocks raise the limit, and because that local system is a Vandermonde matrix in distinct evaluation points it can be solved in ρ^2 rather than ρ^3 time — which is what makes ω=1024 measurable at all.